STATS 101C Week 3 Discussion

Xiaofeng Lin under the direction of Thomas Maierhofer

October 16, 2026

Review: Data Splitting and Cross-Validation

	Question it answers	By hand	mlr3
Data split	How well does the model predict new data? (one noisy estimate)	`sample()`	`partition()`
Cross-validation	Same question, averaged over k splits; also used to choose the model	loop over folds, `cv.glm()`	`rsmp("cv")`, `rsmp("loo")`, `resample()`

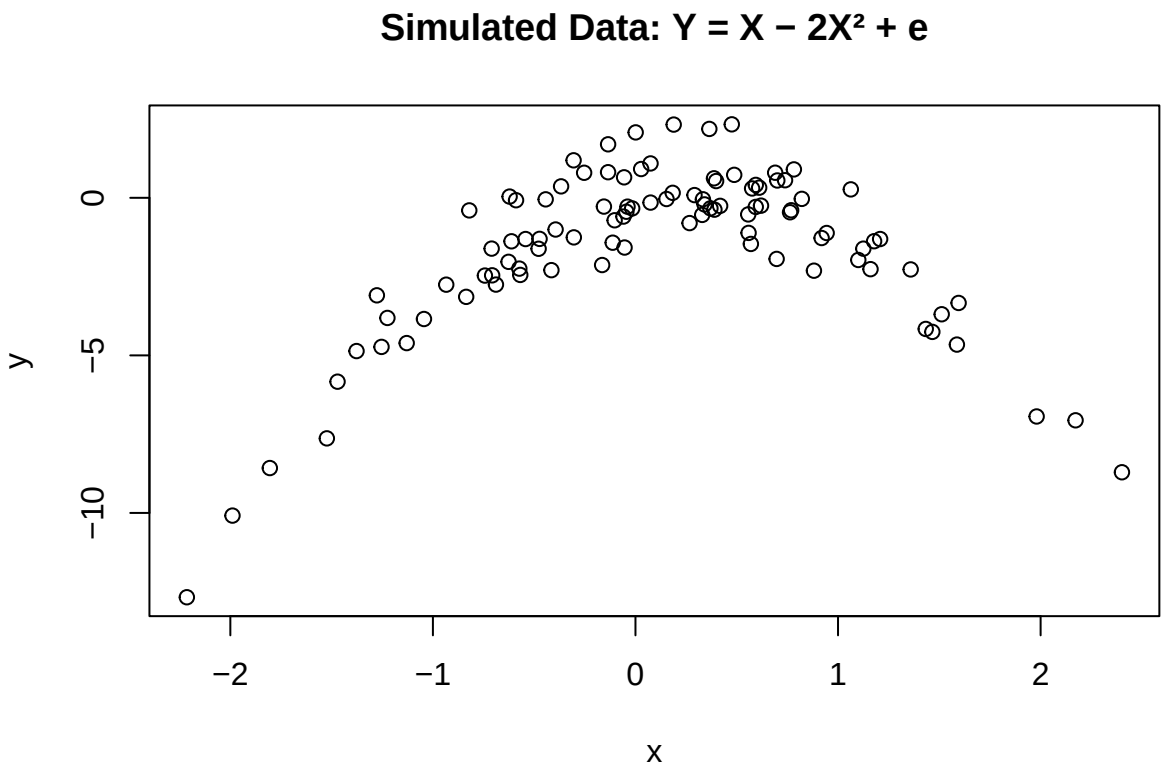
Problem 1: Data Splitting

We use the simulated data from ISLR Chapter 5, Problem 8. Because we simulated it, we know the truth: $Y = X - 2X^2 + \varepsilon$, so the correct model is a quadratic.

```r
set.seed(1)
x <- rnorm(100)
y <- x - 2 * x^2 + rnorm(100)

# Create data frame
sim_data <- data.frame(x = x, y = y)

# Plot the relationship
plot(x, y, main = "Simulated Data: Y = X - 2X² + e")
```



1.1: Split the Data by Hand

Randomly assign 80% of the observations to a training set and the remaining 20% to a test set.

```
set.seed(2)
n_sim <- nrow(sim_data)

# YOUR CODE HERE
train_idx <- sample(seq_len(n_sim), size = floor(0.8 * n_sim))
test_idx <- setdiff(seq_len(n_sim), train_idx)
sim_train <- sim_data[train_idx, ]
sim_test <- sim_data[test_idx, ]
```

1.2: Training Error vs Test Error

For polynomial degrees 1 through 10, fit `y ~ poly(x, d)` **on the training set only**. Record the MSE on the training set and the MSE on the test set.

```
max_degree_sim <- 10

# Data frame to store results: one row per polynomial degree
split_errors <- data.frame(degree = 1:max_degree_sim, train_mse = NA, test_mse = NA)

# YOUR CODE HERE
for (d in 1:max_degree_sim) {
  fit <- lm(y ~ poly(x, d), data = sim_train)
```

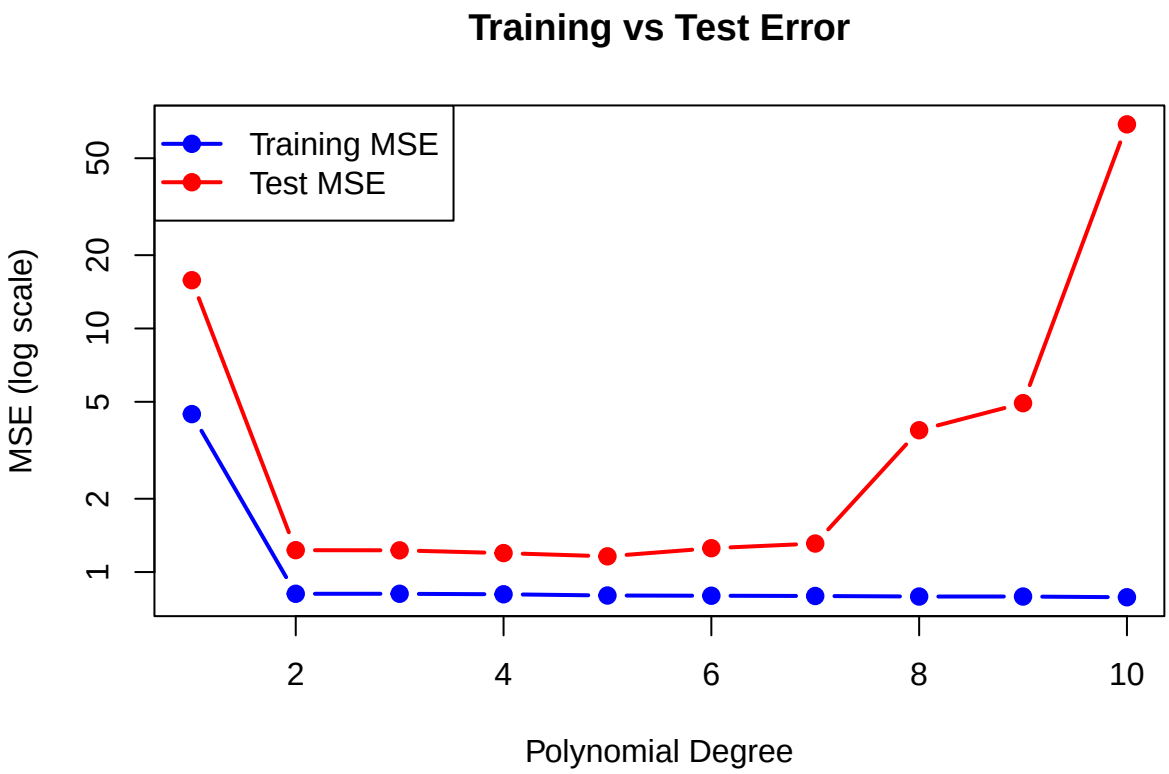
```
  train_predictions <- predict(fit, newdata = sim_train)
  test_predictions <- predict(fit, newdata = sim_test)
  split_errors$train_mse[d] <- mean((sim_train$y - train_predictions)^2)
  split_errors$test_mse[d] <- mean((sim_test$y - test_predictions)^2)
}

split_errors
```

```
##    degree train_mse  test_mse
## 1       1 4.4509819 15.785187
## 2       2 0.8143228  1.229817
## 3       3 0.8142642  1.227919
## 4       4 0.8104977  1.196197
## 5       5 0.8011173  1.160131
## 6       6 0.7993573  1.252658
## 7       7 0.7974228  1.309196
## 8       8 0.7934392  3.821780
## 9       9 0.7933556  4.936457
## 10     10 0.7880980 68.882853
```

Visualize the results (log scale on the y-axis, because the test error gets very large):

```
# Change to eval = TRUE once your loop in 1.2 runs
plot(split_errors$degree, split_errors$train_mse,
     type = "b", log = "y",
     ylim = range(split_errors[, c("train_mse", "test_mse")]),
     xlab = "Polynomial Degree",
     ylab = "MSE (log scale)",
     main = "Training vs Test Error",
     col = "blue", lwd = 2, pch = 19)
lines(split_errors$degree, split_errors$test_mse,
      type = "b", col = "red", lwd = 2, pch = 19)
legend("topleft", legend = c("Training MSE", "Test MSE"),
       col = c("blue", "red"), lwd = 2, pch = 19)
```



Discuss:

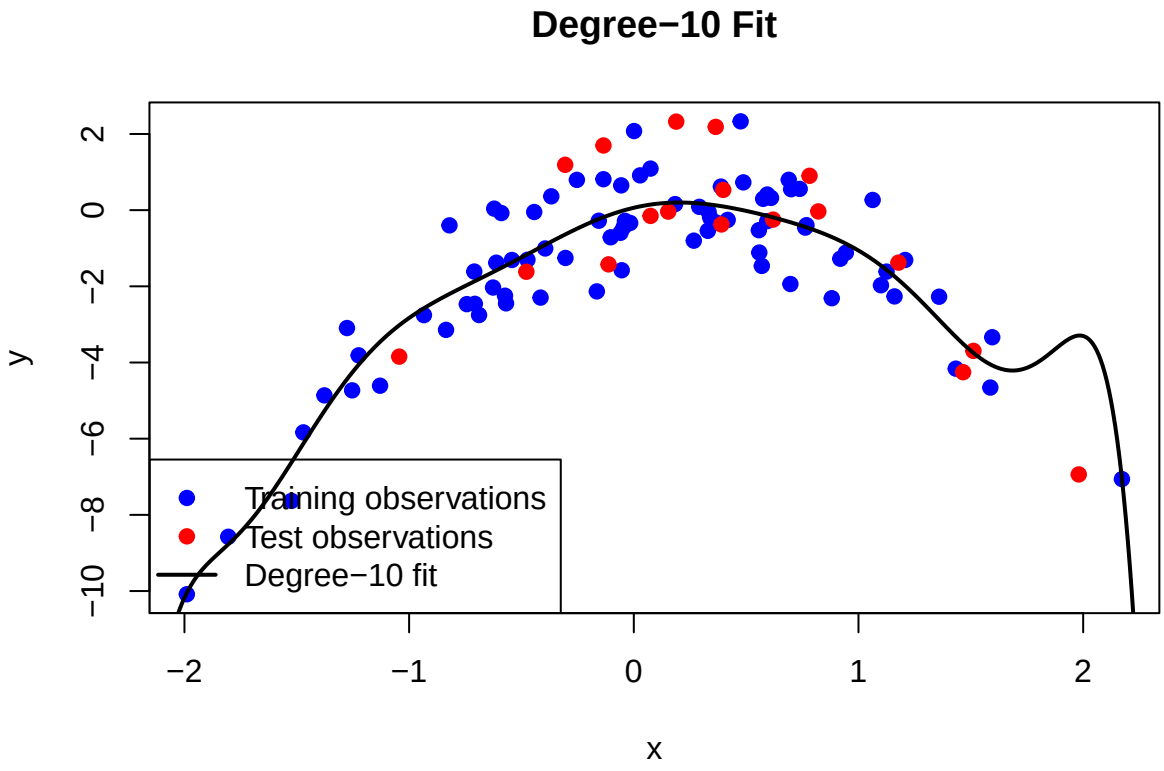
(a) If we chose the model with the smallest *training* MSE, which degree would we pick? Which degree has the smallest *test* MSE? Which degree is the truth?

(b) Why can the training MSE never go up as the degree increases?

(c) Test MSE gets very large at high degrees. Plot the degree-10 fit along with the training and test points. Where do the large test errors come from?

```
# YOUR CODE HERE
degree_10_fit <- lm(y ~ poly(x, 10), data = sim_train)
x_grid <- seq(min(sim_data$x), max(sim_data$x), length.out = 500)
y_grid <- predict(degree_10_fit, newdata = data.frame(x = x_grid))

plot(sim_train$x, sim_train$y,
     xlab = "x", ylab = "y",
     main = "Degree-10 Fit",
     col = "blue", pch = 19)
points(sim_test$x, sim_test$y, col = "red", pch = 19)
lines(x_grid, y_grid, col = "black", lwd = 2)
legend("bottomleft",
       legend = c("Training observations", "Test observations", "Degree-10 fit"),
       col = c("blue", "red", "black"),
       pch = c(19, 19, NA), lty = c(NA, NA, 1), lwd = c(NA, NA, 2))
```



(d) Re-run 1.1 and 1.2 with `set.seed(3)` instead of `set.seed(2)`. Does the test MSE for each degree change? Does the best degree change? If we kept re-splitting and re-checking the test set until we liked the answer, what would be wrong with that?

1.3: Now with mlr3!

mlr3's `regr.lm` learner has no formula interface, so we can't write `poly(x, d)` inside it. Instead we store $x, x^2, \ldots, x^{10}$ as columns and, for degree d, select the first d of them.

```
sim_poly <- data.frame(y = sim_data$y, outer(sim_data$x, 1:max_degree_sim, `^`))
names(sim_poly) <- c("y", paste0("x", 1:max_degree_sim))
head(sim_poly, 3)
```

```
##            y         x1         x2          x3          x4           x5
## 1 -2.0317092 -0.6264538 0.39244438 -0.245848275 0.154012589 -0.0964817733
## 2  0.1583095  0.1836433 0.03372487  0.006193347 0.001137367  0.0002088698
## 3 -3.1431006 -0.8356286 0.69827518 -0.583498718 0.487588224 -0.4074426711
##             x6            x7           x8            x9          x10
## 1 6.044137e-02 -3.786373e-02 2.371988e-02 -1.485941e-02 9.308733e-03
## 2 3.835755e-05  7.044108e-06 1.293603e-06  2.375616e-07 4.362661e-08
## 3 3.404708e-01 -2.845071e-01 2.377423e-01 -1.986642e-01 1.660095e-01
```

Create a regression task and split it with `partition()`. For each degree, train on the training rows, then compute the MSE on the training rows and on the test rows.

```
# YOUR CODE HERE
tsk_sim <- as_task_regr(sim_poly, target = "y")
```

```r
set.seed(2)
sim_partition <- partition(tsk_sim, ratio = 0.8)
mlr3_split_errors <- data.frame(
  degree = 1:max_degree_sim,
  train_mse = NA,
  test_mse = NA
)

for (d in 1:max_degree_sim) {
  degree_task <- tsk_sim$clone(deep = TRUE)
  degree_task$select(paste0("x", 1:d))
  learner <- lrn("regr.lm")
  learner$train(degree_task, row_ids = sim_partition$train)
  train_prediction <- learner$predict(degree_task, row_ids = sim_partition$train)
  test_prediction <- learner$predict(degree_task, row_ids = sim_partition$test)
  mlr3_split_errors$train_mse[d] <- train_prediction$score(msr("regr.mse"))
  mlr3_split_errors$test_mse[d] <- test_prediction$score(msr("regr.mse"))
}

mlr3_split_errors
```

```
##    degree train_mse  test_mse
## 1       1 4.4509819 15.785187
## 2       2 0.8143228  1.229817
## 3       3 0.8142642  1.227919
## 4       4 0.8104977  1.196197
## 5       5 0.8011173  1.160131
## 6       6 0.7993573  1.252658
## 7       7 0.7974228  1.309196
## 8       8 0.7934392  3.821780
## 9       9 0.7933556  4.936457
## 10     10 0.7880980 68.882853
```

Check: With the same seed, the current version of mlr3's `partition()` picks the same rows as `sample()` did in 1.1, so your table should match 1.2 exactly. That also confirms that raw polynomial columns and `poly()` give the same fitted values, although their coefficients differ. If your mlr3 version picks a different split, the numbers will change but the pattern should not.

Discuss: In the mlr3 intro slides, the model was trained on `tsk_cars` and then scored with `lrn_lm$predict(tsk_cars)`. Which column of your table does that number correspond to, and why is it too optimistic?

Problem 2: Hand-Coding 10-Fold Cross-Validation

Context from the Book

In Section 5.3.3, the book uses `cv.glm()` to perform k-fold CV. Today, we'll implement this ourselves to understand what's happening under the hood.

2.1: Create the Folds

Split the 392 observations in the Auto dataset into 10 folds.

```r
set.seed(17)  # Same seed as the book uses for 10-fold CV
n <- nrow(Auto)

# YOUR CODE HERE
fold_ids <- sample(rep(1:10, length.out = n))
table(fold_ids)
```

```
## fold_ids
##  1  2  3  4  5  6  7  8  9 10
## 40 40 39 39 39 39 39 39 39 39
```

2.2: One Fold of Cross-Validation

Let's start by doing CV for just fold 1, fitting a quadratic model like the book does.

Model: mpg ~ poly(horsepower, 2)

```r
# YOUR CODE HERE
fold_1_fit <- lm(mpg ~ poly(horsepower, 2),
                 data = Auto,
                 subset = fold_ids != 1)
fold_1_predictions <- predict(fold_1_fit, newdata = Auto[fold_ids == 1, ])
fold_1_mse <- mean((Auto$mpg[fold_ids == 1] - fold_1_predictions)^2)
fold_1_mse
```

```
## [1] 18.61248
```

2.3: Complete 10-Fold CV

Now extend this to all 10 folds and try different polynomial degrees (1 through 5).

```r
# We'll try polynomial degrees 1 through 5
max_degree <- 5

# Matrix to store results: rows = polynomial degrees, columns = folds
cv_errors <- matrix(NA, nrow = max_degree, ncol = 10)

# YOUR CODE HERE
for (d in 1:max_degree) {
  for (fold in 1:10) {
    cv_fit <- lm(mpg ~ poly(horsepower, d),
                 data = Auto,
                 subset = fold_ids != fold)
    cv_predictions <- predict(cv_fit, newdata = Auto[fold_ids == fold, ])
    cv_errors[d, fold] <- mean((Auto$mpg[fold_ids == fold] - cv_predictions)^2)
  }
}

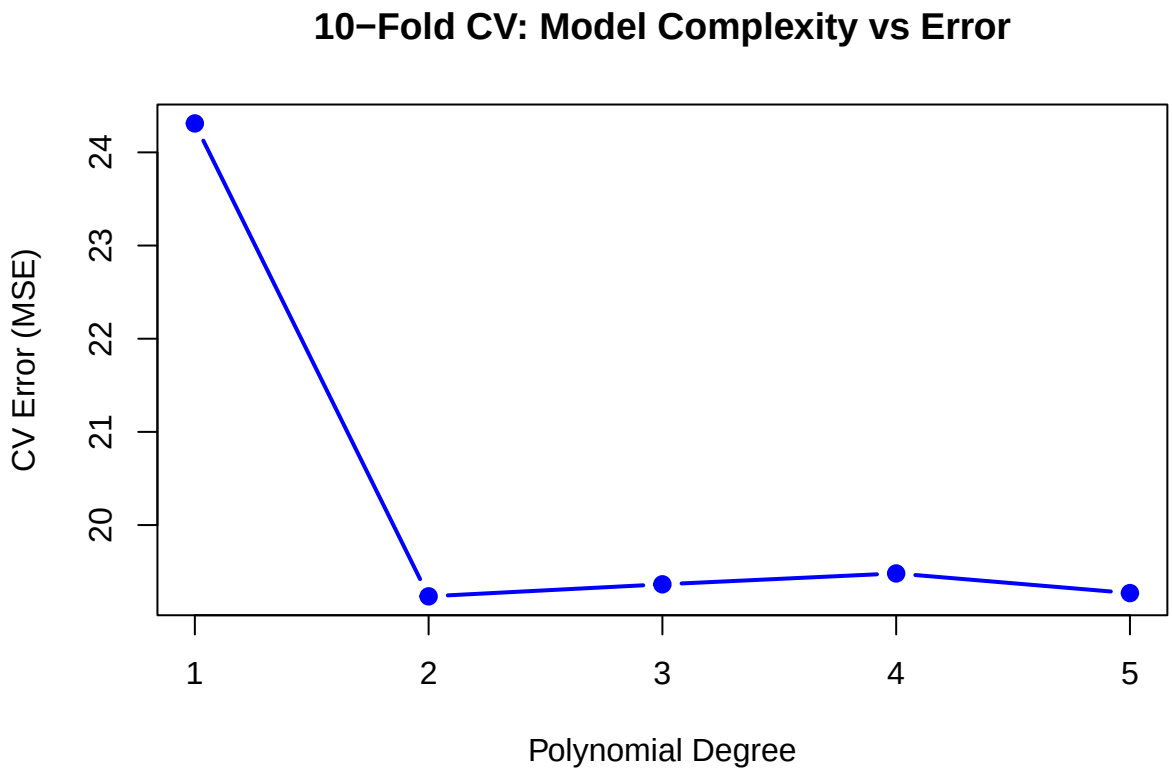
mean_cv_errors <- rowMeans(cv_errors)
data.frame(degree = 1:max_degree, cv_mse = mean_cv_errors)
```

```
##   degree   cv_mse
## 1      1 24.31034
```

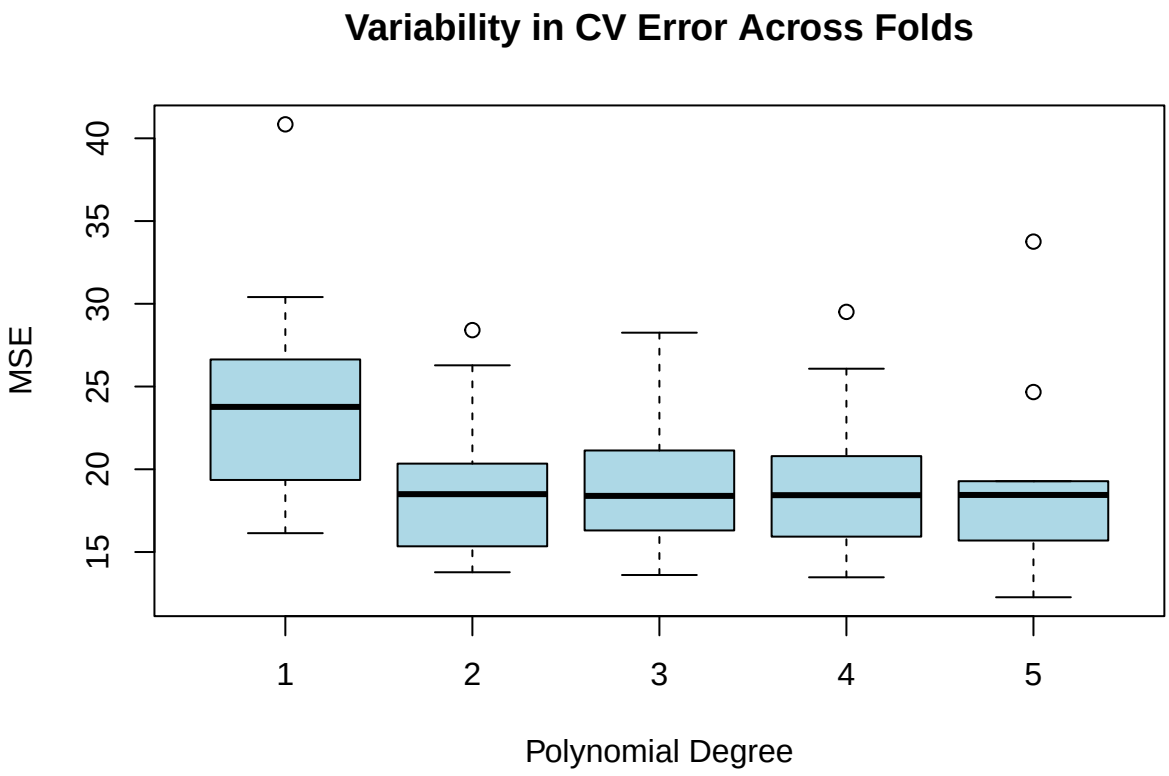
```
## 2      2 19.23502
## 3      3 19.36393
## 4      4 19.48077
## 5      5 19.26978
```

Visualize the results:

```r
# Change to eval = TRUE once your loop in 2.3 runs
# Plot CV error by polynomial degree
plot(1:max_degree, mean_cv_errors,
     type = "b",
     xlab = "Polynomial Degree",
     ylab = "CV Error (MSE)",
     main = "10-Fold CV: Model Complexity vs Error",
     col = "blue", lwd = 2, pch = 19)
```




```r
# Also show the variability across folds for each degree
boxplot(t(cv_errors),
        names = 1:max_degree,
        xlab = "Polynomial Degree",
        ylab = "MSE",
        main = "Variability in CV Error Across Folds",
        col = "lightblue")
```



2.4: Compare with cv.glm()

The book uses `cv.glm()` from the boot package. Let's verify our results. `cv.glm()` draws its own random folds, so its numbers will be close to yours but not identical.

```
# YOUR CODE HERE
set.seed(17)
cv_glm_errors <- numeric(max_degree)

for (d in 1:max_degree) {
  glm_fit <- glm(mpg ~ poly(horsepower, d), data = Auto)
  cv_glm_errors[d] <- cv.glm(Auto, glm_fit, K = 10)$delta[1]
}

cv_comparison <- data.frame(
  degree = 1:max_degree,
  hand_coded_10_fold = mean_cv_errors,
  cv_glm_10_fold = cv_glm_errors
)
cv_comparison
```

```
##   degree hand_coded_10_fold cv_glm_10_fold
## 1      1           24.31034       24.27207
## 2      2           19.23502       19.26909
## 3      3           19.36393       19.34805
```

```
## 4      4           19.48077       19.29496
## 5      5           19.26978       19.03198
```

If you leave out the `K` argument, `cv.glm()` does LOOCV ($K = n$). Compute the LOOCV error for degrees 1 through 5 and add it to the comparison.

```
# YOUR CODE HERE
loocv_glm_errors <- numeric(max_degree)

for (d in 1:max_degree) {
  glm_fit <- glm(mpg ~ poly(horsepower, d), data = Auto)
  loocv_glm_errors[d] <- cv.glm(Auto, glm_fit)$delta[1]
}

cv_comparison$cv_glm_loocv <- loocv_glm_errors
cv_comparison
```

```
##   degree hand_coded_10_fold cv_glm_10_fold cv_glm_loocv
## 1      1           24.31034       24.27207     24.23151
## 2      2           19.23502       19.26909     19.24821
## 3      3           19.36393       19.34805     19.33498
## 4      4           19.48077       19.29496     19.42443
## 5      5           19.26978       19.03198     19.03321
```

2.5: Now with mlr3!

Let's see how much easier this is with mlr3. As in 1.3, we first store the polynomial terms as columns. Use only `mpg` and `horsepower` here. If you passed all of `Auto` to `as_task_regr()`, every other column, including the car `name`, would become a feature.

```
auto_poly <- data.frame(mpg = Auto$mpg, outer(Auto$horsepower, 1:max_degree, `^`))
names(auto_poly) <- c("mpg", paste0("hp", 1:max_degree))
tsk_auto <- as_task_regr(auto_poly, target = "mpg")
tsk_auto
```

```
## <TaskRegr:auto_poly> (392 x 6)
## * Target: mpg
## * Properties: -
## * Features (5):
##   - dbl (5): hp1, hp2, hp3, hp4, hp5
```

Use `rsmp("cv", folds = 10)` and `resample()` to compute the 10-fold CV error for degrees 1 through 5. Instantiate the resampling once so every degree uses the same folds. With `set.seed(17)`, the current version of mlr3 draws the same folds as your code in 2.1, so the numbers should match 2.3 exactly. `cv.glm()` assigns folds differently.

```
# YOUR CODE HERE
set.seed(17)
cv_resampling <- rsmp("cv", folds = 10)
cv_resampling$instantiate(tsk_auto)
mlr3_cv_errors <- numeric(max_degree)

for (d in 1:max_degree) {
  degree_task <- tsk_auto$clone(deep = TRUE)
  degree_task$select(paste0("hp", 1:d))
  cv_result <- resample(
```

```
    degree_task,
    lrn("regr.lm"),
    cv_resampling,
    store_models = FALSE
  )
  mlr3_cv_errors[d] <- cv_result$aggregate(msr("regr.mse"))
}

data.frame(degree = 1:max_degree, cv_mse = mlr3_cv_errors)
```

```
##   degree   cv_mse
## 1      1 24.31034
## 2      2 19.23502
## 3      3 19.36393
## 4      4 19.48077
## 5      5 19.26978
```

2.6 (Optional): LOOCV and Repeated CV in mlr3

For the quadratic model, compute the LOOCV error with `rsmp("loo")` (compare with 2.4) and a 5-times repeated 10-fold CV error with `rsmp("repeated_cv")`.

```
# YOUR CODE HERE
quadratic_task <- tsk_auto$clone(deep = TRUE)
quadratic_task$select(c("hp1", "hp2"))

loocv_result <- resample(
  quadratic_task,
  lrn("regr.lm"),
  rsmp("loo"),
  store_models = FALSE
)

set.seed(17)
repeated_cv_result <- resample(
  quadratic_task,
  lrn("regr.lm"),
  rsmp("repeated_cv", folds = 10, repeats = 5),
  store_models = FALSE
)

data.frame(
  resampling = c("LOOCV", "5 x 10-fold CV"),
  mse = c(
    loocv_result$aggregate(msr("regr.mse")),
    repeated_cv_result$aggregate(msr("regr.mse"))
  )
)
```

```
##       resampling      mse
## 1          LOOCV 19.24821
## 2 5 x 10-fold CV 19.21260
```

Discuss: Which degree would you choose based on CV? How does this compare with what the training MSE in Problem 1 would have told you?